Aggregation, Validation, and Indexes

A request summary can look right and still count the wrong things.

Day 1: a reliable summary and a required date
Day 2: an ordered query and its measured work

The SQL summary we already know

```
SELECT category, COUNT(*) AS active_count
FROM tickets
WHERE status IN ('new', 'open', 'in_progress')
GROUP BY category
ORDER BY category;
```

WHERE selects tickets. GROUP BY changes the unit of the result.
Each output row represents one category.

MongoDB expresses these operations as stages in a list.
The count still depends on what reaches the grouping operation.

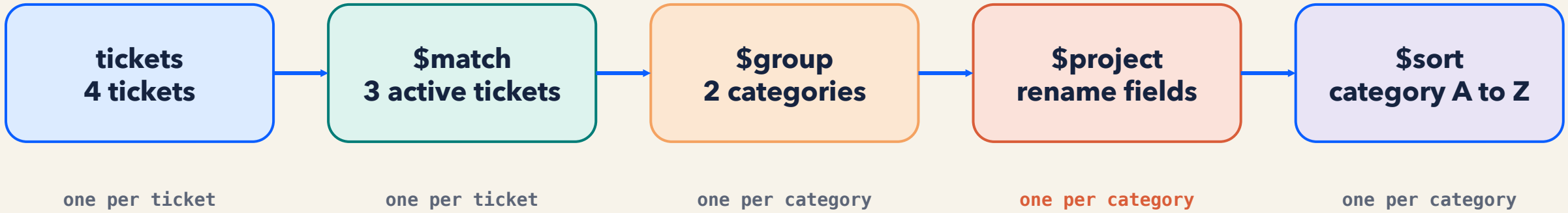
Four tickets for the Day 1 notebook

ID	Category	Status	Priority	Opened (UTC)
1001	streetlight	open	urgent	2026-02-01
1002	sanitation	in_progress	high	2026-02-02
1003	streetlight	resolved	urgent	2026-02-03
1004	streetlight	new	low	2026-02-04

Active IDs: 1001, 1002, 1004. Resolved ticket 1003 stays stored.
This fresh four-ticket case replaces neither the Week 10 fixture nor the CSVs.

DAY 1

The active-ticket summary pipeline




```
4 input documents, then 3 tickets, then 2 category summaries
```

Grain means what one result document represents.

$match keeps the active tickets

```
active_filter = {
    "status": {"$in": ["new", "open", "in_progress"]}
}
matched = list(tickets.aggregate([{"$match": active_filter}]))
print(sorted(row["ticket_id"] for row in matched))
```

```
Result: [1001, 1002, 1004]
```

The collection still contains four tickets.
$match changes this result stream, without deleting stored data.

$group creates one result per category

```
group_stage = {"$group": {
    "_id": "$category",
    "active_count": {"$sum": 1}
}}
grouped = list(tickets.aggregate([
    {"$match": active_filter}, group_stage
]))
```

Grouping expression	Meaning	Result groups
"$category"	Read the category field	sanitation: 1, streetlight: 2
"category"	Use one constant string	category: 3

$project names fields and $sort orders rows

```
pipeline = [
    {"$match": active_filter}, group_stage,
    {"$project": {"_id": 0, "category": "$_id", "active_count": 1}},
    {"$sort": {"category": 1}}
]
pprint(list(tickets.aggregate(pipeline)))
```

category	active_count
sanitation	1
streetlight	2

The grouped _id holds a category value. It is not a ticket ID.

A conditional count and a latest date

```
extended_group = {"$group": {
    "_id": "$category",
    "active_count": {"$sum": 1},
    "urgent_count": {"$sum": {"$cond": [
        {"$eq": ["$priority", "urgent"]}, 1, 0
    ]}},
    "newest_opening": {"$max": "$opened_at"}
}}
```

$cond contributes 1 for urgent, otherwise 0. $max keeps the latest date.
The lab adds these fields to $group and exposes both in $project.

The extended summary has traceable inputs

Category	Active	Urgent	Newest opening
sanitation	1	0	2026-02-02 UTC
streetlight	2	1	2026-02-04 UTC

Streetlight uses tickets 1001 and 1004.
Only 1001 is urgent. Ticket 1004 has the newest opening date.

Ticket 1003 is urgent but resolved, so it contributes to none of these totals.

$unwind expands each event into a row

```
event_rows = list(tickets.aggregate([
    {"$match": active_filter}, {"$unwind": "$events"},
    {"$project": {"_id": 0, "ticket_id": 1,
                  "event_type": "$events.type"}},
    {"$sort": {"ticket_id": 1, "event_type": 1}}
]))
```

ticket_id	event_type
1001	assigned
1001	created
1002	created

Ticket 1004 has events: [] and contributes no rows to this default unwind.

Equal totals can hide different ticket identities

Result being counted	IDs contributing rows	Total
Active tickets	1001, 1002, 1004	3
Events on active tickets	1001, 1001, 1002	3

Even category totals happen to match: streetlight 2, sanitation 1.
The urgent count exposes the error: two event rows, but one urgent ticket.

Preserving empty arrays retains 1004 but still duplicates 1001.
For this ticket summary, leave $unwind out.

A validator describes allowed documents

Rule	What it checks	What remains separate
required	The field exists	Its type and value
bsonType: "date"	The value is a BSON date	Whether the date is plausible
enum: [...]	The value belongs to a list	Whether a status transition is allowed

A focused schema can allow additional fields.
The baseline notebook requires ticket_id, status, and subject.

The baseline schema is a command document

```
validator = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["ticket_id", "status", "subject"],
    "properties": {
        "ticket_id": {"bsonType": ["int", "long"]},
        "status": {"enum": ["new", "open", "in_progress",
                            "resolved", "closed"]},
        "subject": {"bsonType": "string"}
    }
}}
```

Python dictionaries describe MongoDB rules.
A Python variable by itself does not install a server validator.

A required opening date needs two declarations

```
schema = validator["$jsonSchema"]
schema["required"] = ["ticket_id", "status", "subject", "opened_at"]
schema["properties"]["opened_at"] = {"bsonType": "date"}

# Real MongoDB only, after the notebook's Atlas connection.
database.command({"collMod": "tickets", "validator": validator,
                  "validationLevel": "strict",
                  "validationAction": "error"})
```

Editing a dictionary and installing it are different operations.
The notebook skips installation in local mongomock mode.

Allowed input and three counterexamples

Test document	Relevant difference	After the date rule
1099	new + a BSON date	Accept
1100	status: "almost_done"	Reject invalid status
1101	opened_at is date-shaped text	Reject incorrect type
1102	opened_at is absent	Reject missing field

The notebook inserts each test separately and removes its test records.
Atlas executes validation. Local mode supplies a labeled trace to interpret.

Lab 1: a reliable request summary

In Notebook 07, extend the summary and add the date rule.
Use ticket IDs to explain the array-counting error.
Submit one completed notebook in Brightspace, individually.

For Atlas, allow the notebook runtime IPv4 as a temporary /32 entry.
Use the hidden URI prompt and keep TLS verification enabled.

```
database.drop_collection("tickets")
client.close()
```

Drop only this practice collection, then remove its temporary Atlas IP rule.

Day 2: the newest open requests

A queue displays 20 requests.
The database may examine many more.

We will compare the same ordered results before and after an index.
Then you will complete a separate MongoDB University sort lab.

A separate dataset for the live demonstration

Property	Instructor demonstration
Collection	sort_tickets in a new practice database
Dataset	10,000 synthetic tickets with unique IDs
Status distribution	2,000 open and 8,000 resolved
Opening dates	One-minute increments, beginning February 1, 2026 UTC

The four-ticket case explains correctness.
This larger case makes query-plan work visible.

The queue query in mongosh

```
const filter = {status: "open"};
const projection = {
  _id: 0, ticket_id: 1, opened_at: 1, subject: 1
};
const order = {opened_at: -1, ticket_id: -1};
const beforeRows = db.sort_tickets.find(filter, projection)
  .sort(order).limit(20).toArray();
```

Newest date first, then descending ticket ID to resolve tied dates.
A limit caps returned rows, not necessarily examined documents.

Day 2 live code uses mongosh JavaScript. Notebook 07 uses Python.

Compound index order matches this queue

```
db.sort_tickets.createIndex(
  {status: 1, opened_at: -1, ticket_id: -1},
  {name: "open_by_recent"}
);
```

Index position	Role in this query
status: 1	Equality selects the open status range
opened_at: -1	Dates descend within that range
ticket_id: -1	Ticket IDs break ties in that date order

The name does not make this a partial index. It indexes every ticket.

explain reports the work behind the result

```
const after = db.sort_tickets.find(filter, projection)
  .sort(order).limit(20).explain("executionStats");
printjson({
  returned: after.executionStats.nReturned,
  documents: after.executionStats.totalDocsExamined,
  keys: after.executionStats.totalKeysExamined,
  plan: after.queryPlanner.winningPlan
});
```

The same command runs before index creation in the live example.
Compare the normal query results as well as both execution plans.

Measured work on the 10,000-ticket example

Measurement	Before index	After index
Returned documents	20	20
Documents examined	10,000	20
Index keys examined	0	20
Main plan operations	COLLSCAN + SORT	IXSCAN + FETCH

Observed on local MongoDB 8.0.29 with the supplied synthetic data.
Both runs return the same ordered IDs and field values.

These work counts do not establish a 500-times runtime speedup.

FETCH and the cost of keeping an index

The index contains status, opened_at, and ticket_id.
The page also needs subject, so MongoDB fetches the documents.

The extra index occupies storage.
Inserts and changes to indexed values also maintain index entries.

This result uses an index without being a covered query.
Adding every field to every index would add maintenance and storage.

Moving a limit can change the answer

Pipeline on the four-ticket case	Returned ticket IDs
Match open, sort newest, limit 1	1001
Sort newest, limit 1, then match open	None: newest ticket 1004 has status new

Both pipelines run successfully. They answer different questions.
The second checks whether the newest ticket of any status is open.

A filter on a calculated group count also needs that count to exist first.

A faster query can still be incorrect

Mechanism	What the experiment establishes
Aggregation	The summary counts the intended ticket identities
Validation	Specified new inputs pass or fail a document rule
Index and explain	A chosen access path changes measured query work

An index cannot repair a duplicated count or an invalid date.
Installing validation does not rewrite old documents.

Lab 2: sort performance

MongoDB University
Improving Performance of $sort stages (Lab Only)

Open the exact lab link in the Week 11 guide.
Complete it individually and keep one redacted completion image.
This is separate from the instructor's sort_tickets demonstration.

If access fails, use the assignment's labeled plan-comparison fallback.
Include the image or fallback with your video response in Brightspace.

An inventory case and a design recommendation

Building an Event-Driven Inventory Platform: A Case Study
The video link is in Week 11 Lab 2.

Write 200-300 words identifying one demonstrated decision.
Connect a specific moment to a concept from class and explain a tradeoff.
Describe how you would communicate it to a developer or operator.

Separate what the presenter shows from a change you propose.
One Brightspace text submission with the lab image. No Markdown file.

Describing the work in an interview

I traced an aggregation against known ticket IDs and caught a count that duplicated one request while losing another. I tested a required date with both valid and invalid inputs. I also compared an ordered query before and after a compound index, checking that its results stayed the same while the documents examined decreased.

Adapt this account to the work you actually completed.
If you used the local validation trace, say you interpreted the rule.
Next week: replication, consistency, and recovery after data loss.